KG COLLEGE OF ARTS AND SCIENCE
Autonomous Institution | Affiliated to Bharathiar University
Accredited with A++ Grade by NAAC
ISO 9001:2015 Certified Institution
KGiSL Campus, Saravanampatti, Coimbatore – 641 035

JAVA PROJECT REPORT ON

VEHICLE RENTAL BOOKING AND PAYMENT MANAGEMENT SYSTEM

Submitted by

MITHUL KIRUTHIK R S

25BIT125

Under the Guidance of

MUGILAN R

Technical Trainer

Technical Department, KG College of Arts and Science

Academic Year: **2025 – 2026**

APRIL 2026

Semester: **I**

DECLARATION

I hereby declare that the project entitled "**VEHICLE RENTAL BOOKING AND PAYMENT MANAGEMENT SYSTEM"** submitted in partial fulfillment of the requirements for the course [**JAVA PROGRAMMING]** is a **record of my original work** carried out under the guidance of **[MUGILAN R]**, Technical Trainer, Technical Department, KG College of Arts and Science.

I further declare that this project has not been submitted previously, either in part or in full, for the award of any degree, diploma, or other similar title, at this or any other institution.

I have acknowledged all sources of information used in this project wherever applicable.

April 2026

Coimbatore

_____

(Your Name)

CERTIFICATE

This is to certify that the project entitled **"VEHICLE RENTAL BOOKING AND PAYMENT MANAGEMENT SYSTEM"** is a **bona fide work** carried out by **[MITHUL KIRUTHIK R S] [B.SC.IT "B" (2025-2028)]**, **[B.SC.INFORMATION AND TECHNOLOGY]**, **KG College of Arts & Science**, in partial fulfillment of the requirements for the course **[JAVA PROGRAMMING]**.

This project has been completed under my supervision and guidance during the academic period **April 2026**.

Place: COIMBATORE

Date:

Submitted for the Viva-Voce Examination held on _____

Guide:
(Technical Trainer, KGCAS)

Head of the Department
(Technical Head, KGCAS)

TABLE OF CONTENTS

S. No.	Topics	Page. No.
1	**Introduction**	
1.1	Objective	
1.2	Overview	
1.3	Features	
2	**Program Analysis**	
2.1	Problem Definition	
2.2	Project Overview	
2.3	Scopes and Limitations	
3	**System Design**	
3.1	Modules	
4	**Methodology**	
4.1	Problem Analysis	
4.2	System Design	
4.3	Program Development	
4.4	Testing and Validation	
4.5	Result and Documentation	
4.6	Algorithm	
4.7	Flowchart	
5	**Source Code**	
6	**Input and Output Screens**	
7	**Discussion and Conclusion**	
7.1	Discussion	
7.2	Conclusion	
8	**References**	

ABSTRACT

The **Vehicle Rental Booking and Payment Management System** is a Java-based application designed to automate the process of managing vehicle rentals efficiently. In many small rental businesses, operations such as maintaining vehicle records, handling customer details, and calculating rental charges are performed manually. This often leads to calculation errors, poor record management, and difficulty in tracking vehicle availability. This project aims to simplify and streamline the rental process by providing an automated system that manages bookings and calculates payments accurately.

The system allows users to view vehicle details such as vehicle ID, model, type, rental cost per day, and availability status. It also enables users to enter customer details including customer ID, name, and contact number. Based on these inputs, the system facilitates vehicle booking for a specified number of days and automatically calculates the total rental amount using a simple formula.

The application is developed using Core Java programming concepts including classes, objects, methods, loops, conditional statements, and collections like ArrayList. The system is implemented as a console-based application and can be further enhanced using Java Swing or JavaFX to provide a graphical user interface.

The main objective of the system is to improve efficiency in managing vehicle rentals, reduce manual errors in booking and billing, and maintain organized records of vehicles, customers, and transactions. This project demonstrates the practical implementation of object-oriented programming concepts in Java and provides a foundation for developing more advanced rental management systems in the future.

1.INTRODUCTION

In today's fast-paced world, vehicle rental services have become an essential part of transportation systems. Managing rental operations manually can be challenging, as it involves maintaining records of vehicles, customers, bookings, and payments. Traditional methods using paper-based systems often lead to errors, data loss, and inefficient handling of rental activities.

To overcome these challenges, computerized systems play a vital role in automating rental operations. A digital system helps in maintaining accurate records, tracking vehicle availability, managing customer details, and calculating rental charges efficiently. This improves the overall productivity and reliability of the service.

The **Vehicle Rental Booking and Payment Management System** is a Java-based application designed to automate the process of renting vehicles. This system allows users to view available vehicles, add customer details, book vehicles for a specific number of days, and calculate the total rental amount automatically. It also provides functionality to return vehicles and update their availability status.

The system is developed using **Core Java concepts** such as classes, objects, methods, loops, conditional statements, and ArrayList for dynamic data storage. It follows a **menu-driven approach**, making it simple, interactive, and user-friendly.

The main objective of this system is to reduce manual work, ensure accurate billing, manage vehicle bookings efficiently, and demonstrate the practical implementation of Java programming in real-world applications.

1.1 Objective

The main objective of the **Vehicle Rental Booking and Payment Management System** is to develop a simple and efficient application using Core Java that automates the process of vehicle booking, customer management, and rental payment calculation.

The specific objectives of this project are:

- To design and develop a Java-based vehicle rental management system.
- To store and manage vehicle details such as vehicle ID, model, type, rental cost, and availability status.
- To maintain customer information including customer ID, name, and contact details.
- To automate the vehicle booking process based on availability.
- To calculate the total rental amount automatically based on the number of rental days.
- To update vehicle availability after booking and return.
- To reduce manual errors in managing rental records and billing.
- To provide a user-friendly, menu-driven interface for easy interaction.
- To enable viewing, searching, and managing booking details efficiently.
- To demonstrate the implementation of Core Java concepts such as classes, objects, methods, loops, conditional statements, and ArrayList.

1.2 Overview

The **Vehicle Rental Booking and Payment Management System** is a menu-driven application developed using the Java programming language. This system provides a digital platform to manage vehicle rentals, customer details, bookings, and payment calculations efficiently.

The application allows users to view available vehicles with details such as vehicle ID, model, type, rental cost per day, and availability status. It also enables the user to add customer information including customer ID, name, and contact number.

The system supports the vehicle booking process by allowing users to select a vehicle and assign it to a customer for a specified number of days. Based on the rental duration, the program automatically calculates the total rental amount. Once a vehicle is booked, its availability status is updated, and it can be returned later to make it available again.

Additionally, the system provides options to view all vehicles, display booking details, and manage returns efficiently. This reduces manual effort and ensures accurate record management.

The application is implemented as a console-based system using Core Java concepts such as classes, objects, methods, loops, conditional statements, and ArrayList for dynamic data storage.

This project demonstrates the practical application of Java programming in developing real-world systems for vehicle rental management and automated billing.

1.3 Features

The **Vehicle Rental Booking and Payment Management System** includes the following main features:

- **Vehicle Management** – Allows the system to store and display vehicle details such as vehicle ID, model, type, rental cost per day, and availability status.
- **Customer Data Entry** – Enables the user to enter and manage customer details such as customer ID, name, and contact number.
- **Vehicle Booking System** – Allows users to book vehicles based on availability and assign them to customers for a specified number of days.
- **Automatic Payment Calculation** – Calculates the total rental amount automatically based on the vehicle's rent per day and number of rental days.
- **Booking Records Display** – Displays complete booking details including booking ID, customer name, vehicle details, rental duration, and total amount.
- **Vehicle Return Management** – Allows users to return vehicles and updates their availability status accordingly.
- **Menu-Driven Interface** – Provides a simple and user-friendly console interface for easy interaction.
- **Error Reduction** – Minimizes manual errors in booking management and payment calculation.
- **Efficient Data Management** – Uses ArrayList to store and manage vehicle, customer, and booking records dynamically.

2.PROGRAM ANALYSIS

Program analysis is an important phase in software development where the problem is studied in detail and the system requirements are clearly identified. This stage helps in understanding how the system should function and what features are necessary to ensure efficient and accurate operation.

The analysis phase focuses on identifying the limitations of the existing manual vehicle rental system and proposing a computerized solution that improves booking management, vehicle tracking, and payment processing.

2.1 Problem Definition

In many small vehicle rental businesses, operations such as maintaining vehicle records, managing customer details, and calculating rental charges are handled manually using paper records or simple tools. This traditional approach leads to several challenges.

Some common problems include:

- Errors in rental amount calculation
- Difficulty in tracking vehicle availability
- Time-consuming booking and return process
- Poor management of customer records
- Lack of organized and centralized data
- Increased workload for staff
- Risk of data loss and duplication

Manual systems can result in inefficient service, incorrect billing, and poor customer satisfaction. It becomes difficult to handle multiple bookings and maintain accurate records.

Therefore, there is a need for an automated system that can efficiently manage vehicle rentals, track availability, store customer details, and calculate rental payments accurately.

The **Vehicle Rental Booking and Payment Management System** is designed to overcome these problems by providing a computerized solution that automates booking operations, reduces errors, and ensures faster and more reliable service.

The **Vehicle Rental Booking and Payment Management System** is a Java-based application developed to simplify and automate the process of managing vehicle rentals.

The system provides a menu-driven interface where users can perform operations such as adding customer details, viewing available vehicles, booking vehicles, viewing booking records, and returning vehicles.

The application allows users to select a vehicle based on its availability and assign it to a customer for a specific number of days. Based on the rental duration, the system

automatically calculates the total rental amount. Once a vehicle is booked, its availability status is updated to prevent double booking. When the vehicle is returned, the system updates its status back to available.

The system includes the following functionalities:

- Adding and managing customer details
- Displaying available vehicle information
- Booking vehicles based on availability
- Calculating total rental cost automatically
- Viewing all booking records with complete details
- Returning vehicles and updating availability

This project is implemented using Core Java programming concepts such as classes, objects, methods, loops, conditional statements, and ArrayList for dynamic data storage. The application is designed as a console-based program and can be further enhanced using Java Swing or JavaFX for a graphical user interface.

2.3 Scopes and Limitations

Scope

The scope of the **Vehicle Rental Booking and Payment Management System** includes the following:

- Managing vehicle details and availability status
- Maintaining customer information
- Automating the vehicle booking and return process
- Calculating rental charges based on the number of days
- Displaying and managing booking records efficiently
- Demonstrating the practical use of Core Java programming concepts

The system can be used in:

- Small and medium-scale vehicle rental businesses
- Car and bike rental services
- Travel agencies
- Local transport service providers

Limitations

- The system is console-based and does not include a graphical user interface (GUI).
- Data is not stored permanently (no database or file handling).

- Limited to basic functionalities such as booking and returning vehicles.
- Does not support online booking or digital payment integration.
- No multi-user or networking support.
- Security features such as login authentication are not implemented.

Scope

The scope of the Vehicle Rental Booking and Payment Management System includes the following:

- Managing vehicle details and availability status
- Maintaining customer information
- Automating the vehicle booking and return process
- Calculating rental charges based on the number of days
- Displaying and managing booking records efficiently
- Demonstrating the practical use of Core Java programming concepts

The system can be used in:

- Small and medium-scale vehicle rental businesses
- Car and bike rental services
- Travel agencies
- Local transport service providers

Limitations

- The system is console-based and does not include a graphical user interface (GUI).
- Data is not stored permanently (no database or file handling).
- Limited to basic functionalities such as booking and returning vehicles.
- Does not support online booking or digital payment integration.
- No multi-user or networking support.
- Security features such as login authentication are not implemented.

3. SYSTEM DESIGN

System design is the stage where the structure and components of the system are planned before implementation. In this phase, the system is divided into different modules, and the workflow is clearly defined. Proper system design helps in organizing the program efficiently and simplifies development and maintenance.

The Vehicle Rental Booking and Payment Management System is designed as a menu-driven application where users can manage vehicles, customers, bookings, and payments easily.

The system is divided into several modules, each responsible for performing a specific function.

3.1 Modules

The main modules of the Vehicle Rental Booking and Payment Management System are explained below:

1. Vehicle Management Module

This module is responsible for managing vehicle details and availability.

Functions of this module include:

❖ Storing vehicle details such as:
➢ Vehicle ID
➢ Model name
➢ Type (Bike/Car/SUV)
➢ Rent per day

❖ Displaying all available vehicles
❖ Updating vehicle availability status during booking and return

2. Customer Data Entry Module

This module is used to collect and manage customer details.

Functions include:

❖ Accepting customer information such as:
➢ Customer ID

➢ Name
➢ Phone number

❖ Storing customer data using ArrayList
❖ Ensuring proper record management

---

3. Booking Module

This module handles the process of booking vehicles.

Functions include:

❖ Selecting a vehicle based on availability
❖ Selecting a customer for booking
❖ Entering number of rental days
❖ Creating a booking record with unique booking ID
❖ Updating vehicle status to “not available” after booking

---

4. Payment Calculation Module

This module calculates the total rental amount automatically.

Formula used:

❖ Total Amount = Rent per Day × Number of Days

❖ Ensures accurate billing
❖ Eliminates manual calculation errors

5. Display Module

This module is used to display details of vehicles, customers, and bookings.

The displayed information includes:

- Vehicle details (ID, model, type, rent, availability)
- Customer details (ID, name, phone number)
- Booking details (Booking ID, customer name, vehicle, days, total amount)

- Allows users to view all records clearly

6. Search / View Module

This module helps in viewing and managing booking information.

Functions include:

- Displaying all booking records
- Checking vehicle availability
- Viewing customer details

- Ensures easy access to stored data

7. Return Vehicle Module

This module manages the return process of rented vehicles.

Functions include:

- Accepting booking ID
- Identifying the booked vehicle
- Updating vehicle status to available
- Removing the booking record

- Ensures proper tracking of vehicle usage

- **8. Exit Module**
- This module allows the user to safely exit the application.
- Terminates the program execution when selected
- Ensures smooth closing of the system

4.METHODOLOGY

- Methodology describes the process followed to develop the project. It explains the steps taken from understanding the problem to designing, implementing, testing, and documenting the system.
- For the **Vehicle Rental Booking and Payment Management System**, a structured development approach was followed to ensure that the system meets the requirements and functions efficiently.
- The development process includes the following stages:
- Problem Analysis
- System Design
- Program Development
- Testing and Validation
- Result and Documentation

4.1. Problem Analysis

In this phase, the problem is studied and analyzed in detail. The requirements of the system are identified based on the need for efficient vehicle rental management.

The main problems identified in manual rental systems are:

- Manual calculation of rental charges may lead to errors
- Difficulty in tracking vehicle availability
- Time-consuming booking and return process
- Poor management of customer and booking records
- Lack of organized data
- Increased workload for staff
- Based on these problems, the system requirements were defined to develop an automated rental system that simplifies booking, tracking, and billing processes.

4.2. System Design

In this stage, the overall structure of the system is designed before coding begins.

The system is divided into modules such as:

- Vehicle Management Module
- Customer Data Entry Module
- Booking Module
- Payment Calculation Module
- Display Module
- Return Vehicle Module
- Exit Module
- The program flow and structure are designed so that users can easily interact with the system through a menu-driven interface.

4.3. Program Development

After designing the system, the program is implemented using the Java programming language.

The following steps were followed during development:

- Creation of classes such as Vehicle, Customer, and Booking
- Implementation of menu-driven logic
- Writing code for booking and payment calculation
- Handling user input using Scanner
- Storing data using ArrayList
- Displaying vehicle, customer, and booking details
- Core Java concepts such as classes, objects, loops, conditional statements, methods, and collections were used during development.

4.4. Testing and Validation

Testing ensures that the program works correctly and produces the expected results.

Different test cases were used to verify the functionality of the system

EXAMPLE TEXT CASE

TEST CASE	**Customer details stored correctly**
view vehicles	Vehicle list displayed properly
book vehicle	Booking created successfully
Calculate payment	Correct total amount displayed
Return vehicle	Vehicle status updated correctly
Invalid input	Error handled properly

This phase ensures that the program runs without errors and produces accurate results.

4.5. Result and Documentation

After successful testing, the results of the system are documented. The system generates booking details and calculates rental payments based on user input.

The results are demonstrated through program execution showing input and output.

Proper documentation is prepared to explain the design, implementation, and working of the system.

4.6. Algorithm

Algorithm for Vehicle Rental Booking System:

- Start the program.

- Display the menu options.
- Enter user choice.
- If "Add Customer":
  - Enter customer details
  - Store the data
- If "View Vehicles":
  - Display available vehicles
- If "Book Vehicle":
  - Select vehicle
  - Select customer
  - Enter number of days
  - Calculate total amount
  - Store booking details
- If "View Bookings":
  - Display all booking records
- If "Return Vehicle":
  - Enter booking ID
  - Update vehicle availability
  - Remove booking record
- Repeat until user selects exit.
- End the program.

4.7. Flowchart

Students should draw a flowchart showing the program flow.

Example Flow:

Start
↓
Display Menu
↓
Enter Choice
↓
Add Customer / View Vehicles / Book / Return
↓
Process Request
↓
Show Output
↓
Repeat or Exit

↓
End

5.SOURCE CODE

```
import java.util.*;

class Vehicle {
    int vehicleId;
    String model;
    String type;
    double rentPerDay;
    boolean isAvailable;

    Vehicle(int id, String model, String type, double rent) {
        this.vehicleId = id;
        this.model = model;
        this.type = type;
        this.rentPerDay = rent;
        this.isAvailable = true;
    }

    void displayVehicle() {
        System.out.println(vehicleId + " | " + model + " | " + type + " | Rent/Day: " +
rentPerDay + " | Available: " + isAvailable);
    }
}
```

```
class Customer {
    int customerId;
    String name;
    String phone;

    Customer(int id, String name, String phone) {
        this.customerId = id;
        this.name = name;
        this.phone = phone;
    }

    void displayCustomer() {
        System.out.println(customerId + " | " + name + " | " + phone);
    }
}

class Booking {
    int bookingId;
    Vehicle vehicle;
    Customer customer;
    int days;
    double totalAmount;

    Booking(int id, Vehicle v, Customer c, int days) {
        this.bookingId = id;
        this.vehicle = v;
        this.customer = c;
        this.days = days;
        this.totalAmount = v.rentPerDay * days;
```

```
        v.isAvailable = false;
    }

    void displayBooking() {
        System.out.println("Booking ID: " + bookingId);
        System.out.println("Customer: " + customer.name);
        System.out.println("Vehicle: " + vehicle.model);
        System.out.println("Days: " + days);
        System.out.println("Total Amount: " + totalAmount);
    }
}

public class VehicleRentalSystem {

    static ArrayList<Vehicle> vehicles = new ArrayList<>();
    static ArrayList<Customer> customers = new ArrayList<>();
    static ArrayList<Booking> bookings = new ArrayList<>();

    static Scanner sc = new Scanner(System.in);

    static int vehicleCounter = 1;
    static int customerCounter = 1;
    static int bookingCounter = 1;

    public static void main(String[] args) {

            vehicles.add(new Vehicle(vehicleCounter++, "Honda Activa", "Bike", 300));
        vehicles.add(new Vehicle(vehicleCounter++, "Hyundai i20", "Car", 1200));
        vehicles.add(new Vehicle(vehicleCounter++, "Mahindra Scorpio", "SUV", 2500));
```

```
int choice;

do {
    System.out.println("\n===== VEHICLE RENTAL SYSTEM =====");
    System.out.println("1. Add Customer");
    System.out.println("2. View Vehicles");
    System.out.println("3. Book Vehicle");
    System.out.println("4. View Bookings");
    System.out.println("5. Return Vehicle");
    System.out.println("6. Exit");
    System.out.print("Enter choice: ");
    choice = sc.nextInt();

    switch (choice) {
        case 1:
            addCustomer();
            break;
        case 2:
            viewVehicles();
            break;
        case 3:
            bookVehicle();
            break;
        case 4:
            viewBookings();
            break;
        case 5:
            returnVehicle();
```

```
            break;
        case 6:
            System.out.println("Thank you!");
            break;
        default:
            System.out.println("Invalid choice!");
    }

  } while (choice != 6);
}

  static void addCustomer() {
   sc.nextLine();
   System.out.print("Enter Name: ");
   String name = sc.nextLine();
   System.out.print("Enter Phone: ");
   String phone = sc.nextLine();

   Customer c = new Customer(customerCounter++, name, phone);
   customers.add(c);

   System.out.println("Customer Added Successfully!");
}


static void viewVehicles() {
   System.out.println("\n--- Vehicle List ---");
   for (Vehicle v : vehicles) {
      v.displayVehicle();
```

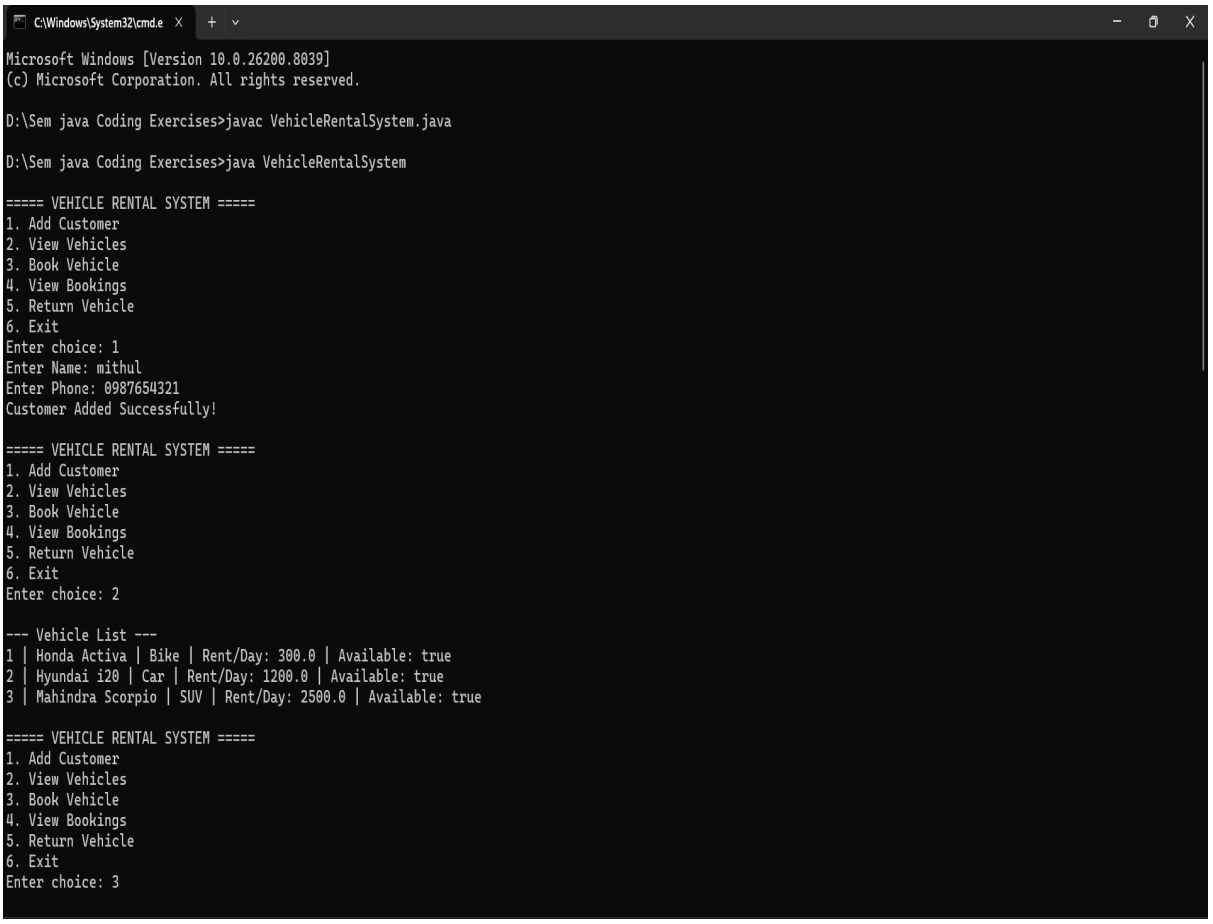
```
    }
}

  static void bookVehicle() {
    if (customers.isEmpty()) {
        System.out.println("No customers found. Add customer first.");
        return;
    }

    viewVehicles();

    System.out.print("Enter Vehicle ID: ");
    int vid = sc.nextInt();

    Vehicle selectedVehicle = null;
    for (Vehicle v : vehicles) {
        if (v.vehicleId == vid && v.isAvailable) {
            selectedVehicle = v;
            break;
        }
    }

    if (selectedVehicle == null) {
        System.out.println("Vehicle not available!");
        return;
    }

    System.out.println("\nSelect Customer:");
    for (Customer c : customers) {
```

```
            c.displayCustomer();
        }

        System.out.print("Enter Customer ID: ");
        int cid = sc.nextInt();

        Customer selectedCustomer = null;
        for (Customer c : customers) {
            if (c.customerId == cid) {
                selectedCustomer = c;
                break;
            }
        }

        if (selectedCustomer == null) {
            System.out.println("Invalid customer!");
            return;
        }

        System.out.print("Enter number of days: ");
        int days = sc.nextInt();

        Booking b = new Booking(bookingCounter++, selectedVehicle, selectedCustomer,
days);
        bookings.add(b);

        System.out.println("Booking Successful!");
        b.displayBooking();
    }
```

```
static void viewBookings() {
    if (bookings.isEmpty()) {
        System.out.println("No bookings found.");
        return;
    }

    System.out.println("\n--- Booking List ---");
    for (Booking b : bookings) {
        b.displayBooking();
        System.out.println("----------------------");
    }
}

  static void returnVehicle() {
    System.out.print("Enter Booking ID to return: ");
    int bid = sc.nextInt();

    Booking found = null;

    for (Booking b : bookings) {
        if (b.bookingId == bid) {
            found = b;
            break;
        }
    }

    if (found == null) {
```

```
            System.out.println("Invalid Booking ID!");
            return;
        }

        found.vehicle.isAvailable = true;
        bookings.remove(found);

        System.out.println("Vehicle Returned Successfully!");
    }
}
```

6.INPUT AND OUTPUT SCREENS



```
C:\Windows\System32\cmd.e

Microsoft Windows [Version 10.0.26200.8039]
(c) Microsoft Corporation. All rights reserved.

D:\Sem java Coding Exercises>javac VehicleRentalSystem.java

D:\Sem java Coding Exercises>java VehicleRentalSystem

===== VEHICLE RENTAL SYSTEM =====
1. Add Customer
2. View Vehicles
3. Book Vehicle
4. View Bookings
5. Return Vehicle
6. Exit
Enter choice: 1
Enter Name: mithul
Enter Phone: 0987654321
Customer Added Successfully!

===== VEHICLE RENTAL SYSTEM =====
1. Add Customer
2. View Vehicles
3. Book Vehicle
4. View Bookings
5. Return Vehicle
6. Exit
Enter choice: 2

--- Vehicle List ---
1 | Honda Activa | Bike | Rent/Day: 300.0 | Available: true
2 | Hyundai i20 | Car | Rent/Day: 1200.0 | Available: true
3 | Mahindra Scorpio | SUV | Rent/Day: 2500.0 | Available: true

===== VEHICLE RENTAL SYSTEM =====
1. Add Customer
2. View Vehicles
3. Book Vehicle
4. View Bookings
5. Return Vehicle
6. Exit
Enter choice: 3
```

```
C:\Windows\System32\cmd.e

Enter choice: 3

--- Vehicle List ---
1 | Honda Activa | Bike | Rent/Day: 300.0 | Available: true
2 | Hyundai i20 | Car | Rent/Day: 1200.0 | Available: true
3 | Mahindra Scorpio | SUV | Rent/Day: 2500.0 | Available: true
Enter Vehicle ID: 2

Select Customer:
1 | mithul | 0987654321
Enter Customer ID: 1
Enter number of days: 3
Booking Successful!
Booking ID: 1
Customer: mithul
Vehicle: Hyundai i20
Days: 3
Total Amount: 3600.0

===== VEHICLE RENTAL SYSTEM =====
1. Add Customer
2. View Vehicles
3. Book Vehicle
4. View Bookings
5. Return Vehicle
6. Exit
Enter choice: 4

--- Booking List ---
Booking ID: 1
Customer: mithul
Vehicle: Hyundai i20
Days: 3
Total Amount: 3600.0
----------------------

===== VEHICLE RENTAL SYSTEM =====
1. Add Customer
2. View Vehicles
3. Book Vehicle
4. View Bookings
```

```
C:\Windows\System32\cmd.e

--- Booking List ---
Booking ID: 1
Customer: mithul
Vehicle: Hyundai i20
Days: 3
Total Amount: 3600.0
----------------------

===== VEHICLE RENTAL SYSTEM =====
1. Add Customer
2. View Vehicles
3. Book Vehicle
4. View Bookings
5. Return Vehicle
6. Exit
Enter choice: 5
Enter Booking ID to return: 1
Vehicle Returned Successfully!

===== VEHICLE RENTAL SYSTEM =====
1. Add Customer
2. View Vehicles
3. Book Vehicle
4. View Bookings
5. Return Vehicle
6. Exit
Enter choice: 6
Thank you!

D:\Sem java Coding Exercises>
```

Output Explanation

Program Start

The system displays a menu with options such as Add Customer, View Vehicles, Book Vehicle, View Bookings, Return Vehicle, and Exit.

Adding Customer (Choice 1)

You entered customer details:

- Name: Ravi
- Phone: 9876543210

The system stores this customer information successfully in the system.

Viewing Vehicles (Choice 2)

The system displays the list of available vehicles with details such as:

- Vehicle ID
- Model name
- Type (Bike/Car/SUV)
- Rent per day
- Availability status

This helps the user choose a vehicle for booking.

Booking Vehicle (Choice 3)

You selected:

- Vehicle ID: 2 (Hyundai i20)
- Customer ID: 1
- Number of days: 3

The system checks vehicle availability and proceeds with booking.

The total amount is calculated using the formula:

Total Amount = Rent per Day × Number of Days

For your input:
Total Amount = 1200 × 3 = 3600

The booking is successfully created, and the vehicle status is updated to **not available**.

Viewing Bookings (Choice 4)

The system displays all booking details including:

- Booking ID
- Customer Name
- Vehicle Model
- Number of Days
- Total Amount

This allows users to view all rental transactions clearly.

Returning Vehicle (Choice 5)

When you enter the Booking ID (e.g., 1), the system:

- Identifies the booking
- Updates the vehicle status to **available**
- Removes the booking record
- A success message is displayed.

Exit (Choice 6)

- The program terminates safely after displaying a thank you message.

7.DISCUSSION & CONCLUSION

7.1 Discussion

The **Vehicle Rental Booking and Payment Management System** was developed to automate the process of managing vehicle rentals, customer details, bookings, and payment calculations. The system allows users to add customer information, view available vehicles, book vehicles, calculate rental charges, and return vehicles efficiently.

During the development process, Core Java programming concepts such as classes, objects, methods, loops, conditional statements, and ArrayList were applied effectively. The system successfully handles vehicle availability, booking operations, and automatic calculation of rental amounts based on the number of days.

The system was tested with different inputs to ensure accuracy and reliability. The results show that the system reduces manual effort, minimizes errors in booking and billing, and improves efficiency in managing rental operations.

7.2 Conclusion

The **Vehicle Rental Booking and Payment Management System** successfully demonstrates the practical application of Core Java programming concepts in developing a real-world application.

The system automates the vehicle rental process, ensures accurate calculation of rental charges, and provides a structured way to manage vehicles, customers, and bookings. It simplifies rental operations and improves efficiency in handling records.

This project helps in understanding how Java can be used to build practical software systems for rental management. The system can be further enhanced by adding features such as database connectivity, graphical user interface (GUI), online booking, payment integration, and multi-user access.

REFERENCES

The following resources were used during the development of this project:

Books

- Herbert Schildt – *Java: The Complete Reference*
- E. Balagurusamy – *Programming with Java*

Websites

- https://www.oracle.com/java
- https://www.geeksforgeeks.org
- https://www.javatpoint.com
- https://www.w3schools.com/java